🧠

fMRI GLM 분석 정리 (SPM Auditory Dataset)

⭐ 핵심 개념, 알아야 하는 포인트

- MRI는 4D 데이터다
- voxel은 뇌의 작은 3D 픽셀이다
- BOLD는 신경활동의 간접 지표다
- event 파일은 자극 타이밍 정보다
- GLM은 자극이 BOLD를 설명하는지 검정한다
- HRF는 자극 후 지연된 혈류 반응을 모델링한다
- design matrix는 GLM의 설명변수 표다
- contrast는 특정 조건 효과를 꺼내는 비교다
- z-map은 activation의 통계적 강도를 보여준다
- mask는 분석할 voxel 범위를 제한한다
- MNI 좌표는 표준 뇌 위치 표현이다

딱 8개만 제대로 이해해야 할 부분

1. `fetch_spm_auditory()`
2. `nib.load()`와 NIfTI 구조
3. `events.tsv` 해석
4. `concat_imgs()`로 4D 만들기
5. voxel 시계열 추출
6. HRF 개념
7. `FirstLevelModel`과 design matrix
8. `compute_contrast()`로 z-map 해석

설명 가능해야 하는 포인트

- 왜 GLM이 필요한가
- 왜 HRF를 써야 하는가
- 왜 mask가 필요한가
- 왜 voxel 하나만으로 결론 내리면 안 되는가
- 왜 activation 위치를 MNI로 해석하는가

그냥 찾아보면 되는 것

- 함수 이름
- 파라미터 세부값
- plotting 옵션

---

1. `datasets.fetch_spm_auditory()`

- nilearn은 뇌영상 데이터를 다루는 파이썬 라이브러리
- datasets.fetch_spm_auditory() 예제용 fmri 데이터
- **SPM = Statistical Parametric Mapping : 뇌 영상 데이터를 통계적으로 분석해서 "어디가 활성화됐는지" 지도(map)로 보여주는 방법**
- fetch 함수 → data.func, data.event 반환하는데 각각 BOLD 시그널, 실험 자극 타이밍 정보를 의미

2. `nibabel` 로 NIfTI 읽기

- NIfTI 파일이란 뇌영상 파일 포맷
- .nii, .nii.gz
- 3D 는 한 시점의 뇌 볼륨 → 시간축 더해지면 4D
- data.func의 여러 3D를 concat_imgs()로 합쳐서 4D로 만들 것
- img.shape → (x, y, z) → 공간 차원 → 4D면 (x, y, z, t)

⭐📂⭐⭐⭐⭐⭐

```
first_img.affine
nib.affines.voxel_sizes(first_img.affine)
```

- **affine matrix = voxel index 좌표 → 실제 mm 좌표로 바꿔줌 (매우 중요)**
- 한 voxel이 실제로 몇 mm인지

3. header와 TR 이해

```
header.get_zooms()
```

- header ? 이미지 메타 데이터 → 복셀 크기, 차원 정보
- TR은 repetition time 한 볼륨을 찍는 시간 ⭐
  - 시간축 간격이 됨
  - GLM 설계의 기본이 됨

💡 **실습 포인트: 코드에서 TR이 애매했던 이유**

- 볼륨 하나에는 time dimension이 없을 수 있어서
- TR을 header에서 바로 못 읽는 경우가 있음.
- 즉 **3D 파일과 4D 파일의 차이**를 이해해야 함.

4. events.tsv 읽기

```
events = pd.read_csv(data.events, sep='\t')
```

- 실험 자극의 타이밍 info → 이벤트 타이밍 단위 확인 필수
- onset, duration, trial_type(조건 이름)

💡 **실습 포인트: 코드에서 TR이 애매했던 이유**

- 내 코드에서 조건의 이름은 listening
- 조건 이름은 실제 데이터를 꼭 확인
- unique(), value_counts()

- onset의 단위: 초 단위인지, scan index인지 확인해야 함
- implication baseline → 자극 없을 때 베이스라인

5. image.concat_imgs()와 4D 시계열

```
fmri_4d = image.concat_imgs(fmri_files)
```

- 여러 3D → 4D (시간축)
- fmri_data = fmri_4d.get_fdata() → shape는 보통 (x,y,z,t)
- 특정 복셀의 시간 변화는 [x, y, z, :]
- mean over time:
  - 평균 구조처럼 보이는 볼륨 생성

```
mean_vol = fmri_data.mean(axis=3)
```

6. BOLD 시계열과 voxel 선택

```
bold_signal = fmri_data[bx, by, bz, :]
```

- 혈중 산소 변화 기반 fMRI 신호
- voxel 하나 시계열의 의미 → 잡음 많을 수도 있음

> 💡 실습 → 복셀 선택한 방식?
>
> - 뇌 안 후보 찾기
> - variance 큰 voxel 선택
>
> 이건 탐색용으로는 가능하지만, **반드시 task-relevant voxel은 아님**.

- 신호 변화가 크다고 의미 있는 voxel은 아님
- noise/motion/경계 artifact일 수도 있음

**시계열 plot도 필요

- 시간축 이해
- 이벤트 타이밍과 대략 맞는지 보기
- 데이터가 정상적으로 읽혔는지 확인

7. `matplotlib` 시각화

```
ax.axvspan(...)
ax.plot(times,bold_signal)
```

- 자극 구간과 시계열을 함께 보기 위해
- 자극과 반응의 관계를 직관적으로 보기 위해

8. HRF란

👉 **뇌의 "표준 반응 곡선"**

👉 **자극 타이밍을 그대로 쓰지 않고, "뇌가 실제로 반응하는 형태"로 바꿔서 모델에 넣는다**

- hemodynamic response function
- 자극 후에 볼드 신호가 천천히 올라갔다가 내려오는 형태
- 자극 시작과 BOLD peak 일치 X
- 이벤트를 HRF랑 합침(convolution)
- GLM에서 왜 `hrf_model="spm"` 을 쓰는지
  - SPM 스타일의 표준 HRF 사용
  - expected BOLD response를 모델링

  이건 fMRI GLM의 핵심 개념

👉 **HRF convolution = 자극을 "뇌가 실제로 반응하는 시간 패턴"으로 변환하는 과정**

9. GLM

General Linear Model

→ HRF 형태로 반응했는가? 를 보는 거

```
glm = FirstLevelModel(...)
glm.fit(fmri_4d, events=events)
```

- BOLD 신호를 여러 설명변수로 설명하는 통계모델
- first-level analysis
  - 개인 한 명의 fMRI 분석
  - subject-level analysis
  - t_r=7.0 (시간축 해석)
  - `hrf_model="spm"` **자극을 HRF 반응 형태로 변환**
  - `drift_model="cosine"` 천천히 변하는 baseline 잡음 제거
  - `high_pass=0.01` 느린 드리프트 제거
  - `smoothing_fwhm=6.0` 공간 smoothing
  - `slice_time_ref`
- fit이 하는 일
  - design matrix 생성 → GLM이 BOLD를 설명하기 위해 쓰는 예측변수 표
  - `design_matrix = glm.design_matrices_[0]`
  - voxel별 회귀분석 수행

```
Y = Xβ + ε
```

- Y: 실제 BOLD
- X: design matrix
- β: 각 설명변수 효과
- ε: 오차

💡 "학습"은 실제 BOLD로 하고, 계수(베타)를 계산
👉 GLM → 실제 BOLD = β × (예상 BOLD) + 노이즈 → 베타 찾기
BOLD ≈ β × predictor

> "이 뇌 신호가
> 👉 내가 예상한 반응 패턴(HRF 적용된 자극)이랑
> 👉 얼마나 비슷하냐?"

10. design matrix

> "이 뇌 신호는
>
> 👉 자극 때문인지
>
> 👉 아니면 느린 드리프트 때문인지
>
> 👉 아니면 그냥 평균 때문인지 설명해보자"
>
> 그 "설명 후보들"을 모아놓은 게 design matrix

- `listening`: 자극 predictor
- `drift_1`, `drift_2`...: 느린 잡음 보정
- `constant`: 평균 baseline
- 자극 효과 말고도 신호에 섞인 느린 변화가 많기 때문
- noise를 모델에 넣어야 자극 효과를 더 정확히 추정 가능

핵심 질문

- `β_listening`이 유의한가?
- 즉 listening 조건이 BOLD를 설명하느냐를 묻는 것.

11. contrast

어느 voxel이 자극과 관련 있는가를 지도 형태로 보여줌.

디자인 메트릭스

listening → 우리가 관심 있는 것
drift_1 → 노이즈

drift_2 → 노이즈
constant → baseline

👉 근데 우리가 궁금한 건: "listening 때문인가?"

👉 필요 없는 건 빼고

👉 원하는 것만 꺼냄 contrast = [1, 0, 0, 0]

```
z_map=glm.compute_contrast("listening",output_type="z_score")
```

- design matrix의 특정 효과를 꺼내는 비교식
- `z_score`: 통계적 유의성 지도
- `effect_size`: 효과 크기 지도
- 양의 z가 클수록 listening에서 activation 증가
- 음의 z는 반대

---

12. mask

EPI = Echo Planar Imaging

👉 뇌를 "엄청 빠르게" 찍기 위한 MRI 촬영 방식

```
from nilearn.masking import compute_epi_mask
#EPI 데이터 기반으로 뇌 영역 추출

mask=compute_epi_mask(fmri_4d)
```

- GLM을 뇌 전체가 아니라 **분석할 만한 뇌 voxel에만 적용하기 위해**
- `mask_img=False`로 했을 때 배경, skull 바깥, 의미 없는 영역까지 분석에 들어가 noise가 많아짐
- `compute_epi_mask` EPI 평균 신호 기반 자동 brain mask

---

13. `plot_stat_map`, `plot_glass_brain`, `plot_roi`

- `plot_roi` 마스크 확인용

- `plot_stat_map`
  - activation map을 slice 위에 그림
  - 실제 위치 확인에 더 좋음
- `plot_glass_brain`
  - 3D 결과를 투명 뇌 윤곽에 투영
  - 전체 패턴 보기 좋음
  - 대신 위치가 약간 덜 정확해 보일 수 있음

---

14. mean EPI 위에 overlay

- 시간 전체에 걸쳐서 평균낸 fMRI 이미지 (시간 기준)

```
mean_epi=image.mean_img(fmri_4d)
plot_stat_map(z_map,bg_img=mean_epi, ...)
```

- 표준 template보다 subject 자신의 functional 배경에 가까워서
- "mask가 이상한가?" "activation 위치가 어긋난 것 같은가?" 확인할 때 매우 유용함

---

15. MNI 좌표

```
peak_coord=find_xyz_cut_coords(z_map)
```

- 표준 뇌 좌표계
- 뇌 위치를 mm 단위로 표현
  - x: 좌(-), 우(+)
  - y: 뒤(-), 앞(+)
  - z: 아래(-), 위(+)
- `(29, 38, 23)` 같은 건 voxel index
- `[-51, -4, 38]` 같은 건 실제 mm 좌표

이 차이 꼭 이해해야 함.

---

16. “좋은 결과”의 기준

- task와 맞는 영역이 뜨는가
- 좌우 대칭성/해부학적 타당성 있는가
- 배경 noise가 과하지 않은가